

PROVA 1 - Processamento Digital de Imagens

Questão 1: Revelando uma foto antiga. O processamento de histograma.

Objetivo: recuperar detalhes de uma imagem com contraste ruim usando transformações de intensidade baseadas no histograma.

Parte I: Equalização

A equalização de histograma redistribui os níveis de cinza de modo que a CDF da imagem resultante se aproxime de uma reta, espalhando o contraste por toda a faixa dinâmica disponível.

```
In [71]: # Importações básicas
import numpy as np
from PIL import Image          # apenas para leitura/gravação de imagens
import matplotlib.pyplot as plt
```

(a) Funções de equalização implementadas manualmente

Implementação do histograma normalizado $p_r(r_k)$, da CDF e da transformação $s_k = (L - 1) \cdot \text{CDF}(r_k)$.

```
In [72]: def calcular_histograma(img):
    """
    Calcula o histograma (contagem absoluta) de uma imagem em tons de cinza (8 bits).
    Percorre cada pixel e incrementa o contador do nível correspondente.
    """
    hist = np.zeros(256, dtype=np.int64) # 256 níveis possíveis (0..255)
    linhas, colunas = img.shape
    for i in range(linhas):
        for j in range(colunas):
            hist[img[i, j]] += 1          # incrementa a contagem do nível r_k
    return hist

def histograma_normalizado(hist, num_pixels):
    """
    Calcula o histograma normalizado  $p_r(r_k) = n_k / N$ ,
    onde  $n_k$  é a contagem do nível k e N é o total de pixels.
    Resulta em uma estimativa da PDF discreta.
    """
    return hist.astype(np.float64) / num_pixels

def calcular_cdf(pr):
    """
    Calcula a CDF (função de distribuição acumulada) a partir do
    histograma normalizado p_r.  $\text{CDF}(r_k) = \sum_{j=0}^k p_r(r_j)$ .
    """
    cdf = np.zeros(256, dtype=np.float64)
    cdf[0] = pr[0]
    for k in range(1, 256):
        cdf[k] = cdf[k - 1] + pr[k]      # acumula as probabilidades
    return cdf

def equalizar_histograma(img):
```

```

"""
Realiza a equalização de histograma completa:
1. Calcula o histograma da imagem.
2. Normaliza para obter  $p_r(r_k)$ .
3. Calcula a CDF.
4. Aplica a transformação  $s_k = \text{round}((L-1) * \text{CDF}(r_k))$ .
5. Mapeia cada pixel da imagem original para o novo valor  $s_k$ .
Retorna a imagem equalizada e dados intermediários.
"""
L = 256                                # número de níveis de cinza
linhas, colunas = img.shape
num_pixels = linhas * colunas          # total de pixels N

# Passo 1: histograma absoluto
hist = calcular_histograma(img)

# Passo 2: histograma normalizado (PDF estimada)
pr = histograma_normalizado(hist, num_pixels)

# Passo 3: CDF acumulada
cdf = calcular_cdf(pr)

# Passo 4: transformação  $s_k = \text{round}((L-1) * \text{CDF}(r_k))$ 
sk = np.round((L - 1) * cdf).astype(np.uint8)

# Passo 5: construir imagem de saída mapeando cada pixel
img_eq = np.zeros_like(img)
for i in range(linhas):
    for j in range(colunas):
        img_eq[i, j] = sk[img[i, j]] # aplica o mapeamento

return img_eq, hist, pr, cdf, sk

```

(b) Aplicação nas imagens de baixo contraste

Carregamos `pollen-lowcontrast.tif` e `messi_lowcontrast.tif` e aplicamos a equalização.

```

In [73]: # Carregar as duas imagens em escala de cinza
img_pollen = np.array(Image.open('pollen-lowcontrast.tif').convert('L')) # garante 8 bits cinza
img_messi = np.array(Image.open('messi_lowcontrast.tif').convert('L'))

print(f'pollen - shape: {img_pollen.shape}, dtype: {img_pollen.dtype}')
print(f'messi - shape: {img_messi.shape}, dtype: {img_messi.dtype}')

```

```

pollen - shape: (500, 500), dtype: uint8
messi - shape: (817, 1080), dtype: uint8

```

```

In [74]: # Equalização da imagem pollen
pollen_eq, pollen_hist, pollen_pr, pollen_cdf, pollen_sk = equalizar_histograma(img_pollen)

# Equalização da imagem messi
messi_eq, messi_hist, messi_pr, messi_cdf, messi_sk = equalizar_histograma(img_messi)

print('Equalização concluída para ambas as imagens.')

```

Equalização concluída para ambas as imagens.

Visualização da CDF e da transformação s_k

In [75]:

```
# Plotar CDF e mapeamento s_k para cada imagem
fig, axes = plt.subplots(2, 2, figsize=(12, 8))

# --- Pollen ---
axes[0, 0].plot(pollen_cdf, color='steelblue', linewidth=1.5)
axes[0, 0].set_title('CDF - Pollen')
axes[0, 0].set_xlabel('Nível de cinza $r_k$')
axes[0, 0].set_ylabel('CDF$(r_k)$')
axes[0, 0].set_xlim([0, 255])
axes[0, 0].grid(True, alpha=0.3)

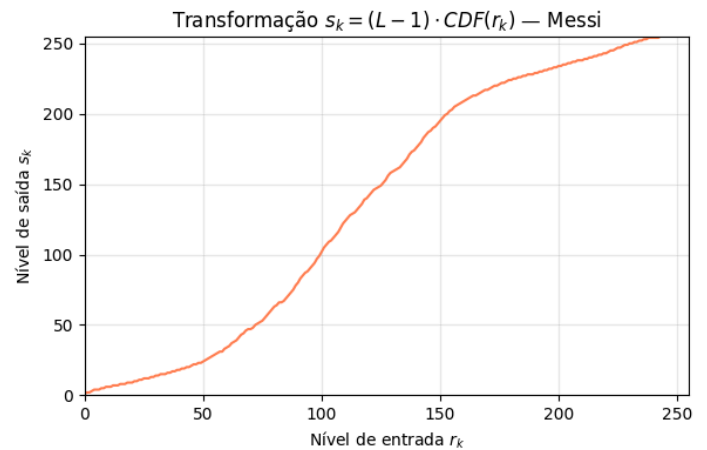
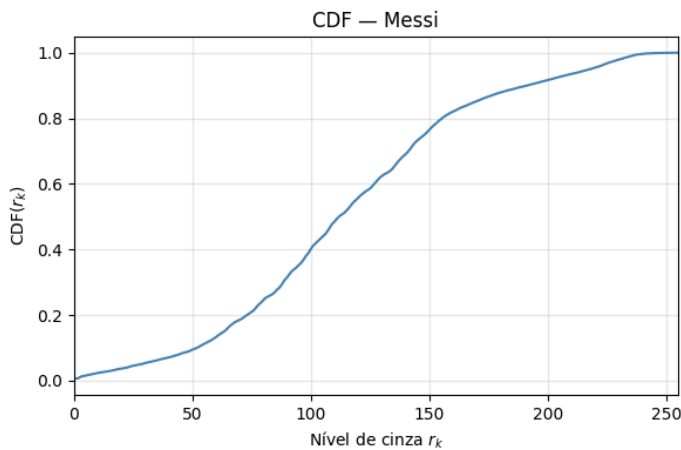
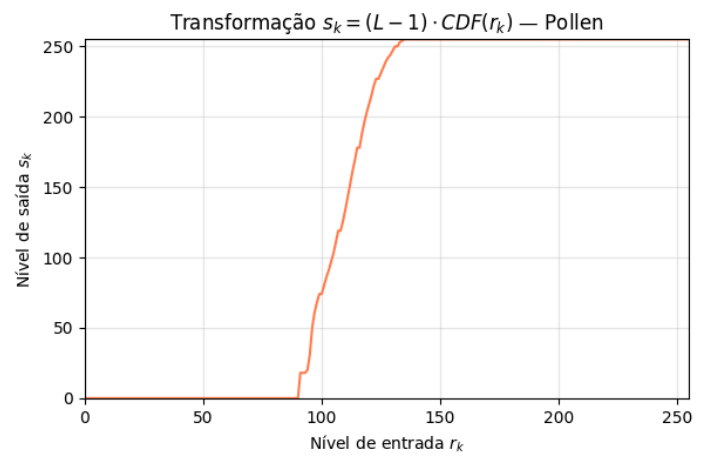
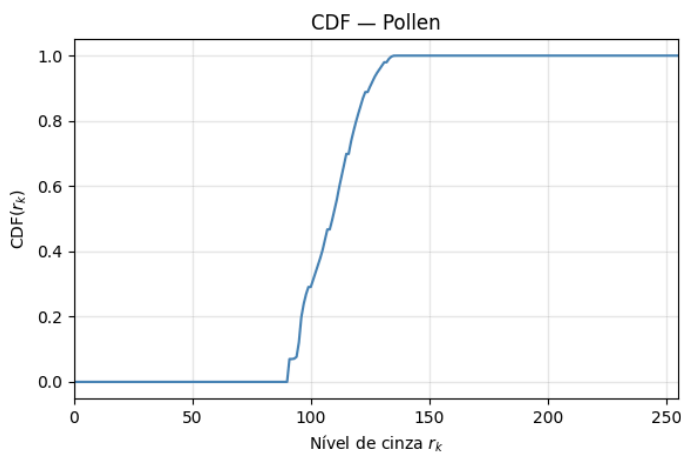
axes[0, 1].plot(pollen_sk, color='coral', linewidth=1.5)
axes[0, 1].set_title('Transformação $s_k = (L-1) \cdot \text{CDF}(r_k)$ - Pollen')
axes[0, 1].set_xlabel('Nível de entrada $r_k$')
axes[0, 1].set_ylabel('Nível de saída $s_k$')
axes[0, 1].set_xlim([0, 255])
axes[0, 1].set_ylim([0, 255])
axes[0, 1].grid(True, alpha=0.3)

# --- Messi ---
axes[1, 0].plot(messi_cdf, color='steelblue', linewidth=1.5)
axes[1, 0].set_title('CDF - Messi')
axes[1, 0].set_xlabel('Nível de cinza $r_k$')
axes[1, 0].set_ylabel('CDF$(r_k)$')
axes[1, 0].set_xlim([0, 255])
axes[1, 0].grid(True, alpha=0.3)

axes[1, 1].plot(messi_sk, color='coral', linewidth=1.5)
axes[1, 1].set_title('Transformação $s_k = (L-1) \cdot \text{CDF}(r_k)$ - Messi')
axes[1, 1].set_xlabel('Nível de entrada $r_k$')
axes[1, 1].set_ylabel('Nível de saída $s_k$')
axes[1, 1].set_xlim([0, 255])
axes[1, 1].set_ylim([0, 255])
axes[1, 1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

```
<>:13: SyntaxWarning: "\c" is an invalid escape sequence. Such sequences will not work in the future. Did you mean "\\c"?
A raw string is also an option.
<>:29: SyntaxWarning: "\c" is an invalid escape sequence. Such sequences will not work in the future. Did you mean "\\c"?
A raw string is also an option.
<>:13: SyntaxWarning: "\c" is an invalid escape sequence. Such sequences will not work in the future. Did you mean "\\c"?
A raw string is also an option.
<>:29: SyntaxWarning: "\c" is an invalid escape sequence. Such sequences will not work in the future. Did you mean "\\c"?
A raw string is also an option.
C:\Users\Pedro\AppData\Local\Temp\ipykernel_43988\3597342116.py:13: SyntaxWarning: "\c" is an invalid escape sequence. Such sequences will not work in the future. Did you mean "\\c"? A raw string is also an option.
    axes[0, 1].set_title('Transformação $s_k = (L-1) \cdot \text{CDF}(r_k)$ - Pollen')
C:\Users\Pedro\AppData\Local\Temp\ipykernel_43988\3597342116.py:29: SyntaxWarning: "\c" is an invalid escape sequence. Such sequences will not work in the future. Did you mean "\\c"? A raw string is also an option.
    axes[1, 1].set_title('Transformação $s_k = (L-1) \cdot \text{CDF}(r_k)$ - Messi')
```



(c) Comparação lado a lado: original × equalizada

In [76]:

```
# --- Comparação para POLLEN ---
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
fig.suptitle('Equalização de Histograma — Pollen', fontsize=15, fontweight='bold')

# Imagem original
axes[0, 0].imshow(img_pollen, cmap='gray', vmin=0, vmax=255)
axes[0, 0].set_title('Imagem Original')
axes[0, 0].axis('off')

# Histograma original
axes[0, 1].bar(range(256), pollen_hist, color='gray', width=1.0)
axes[0, 1].set_title('Histograma Original')
axes[0, 1].set_xlabel('Nível de cinza')
axes[0, 1].set_ylabel('Número de pixels')
axes[0, 1].set_xlim([0, 255])

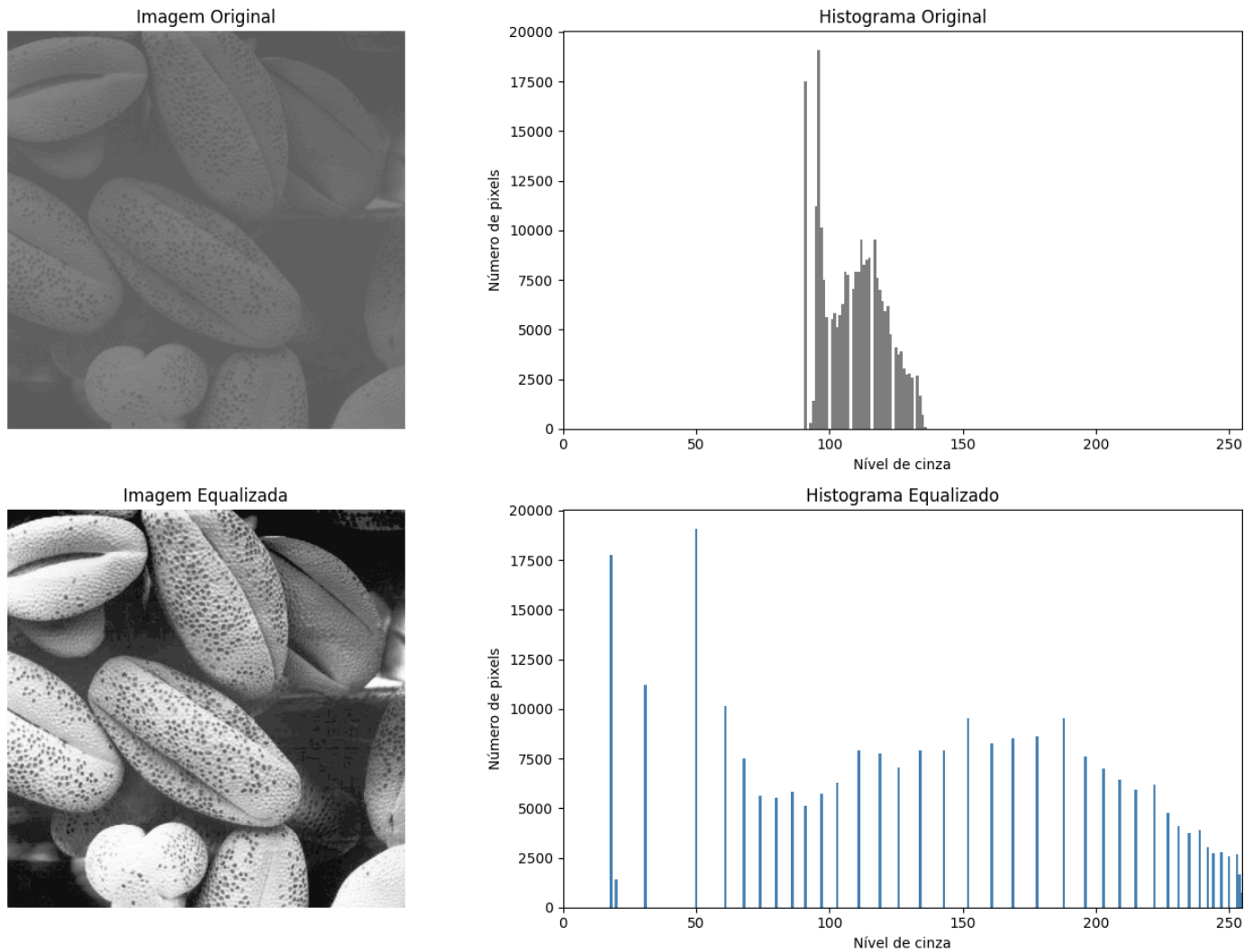
# Imagem equalizada
axes[1, 0].imshow(pollen_eq, cmap='gray', vmin=0, vmax=255)
axes[1, 0].set_title('Imagem Equalizada')
axes[1, 0].axis('off')

# Histograma equalizado
hist_pollen_eq = calcular_historama(pollen_eq) # calcula histograma da imagem resultado
axes[1, 1].bar(range(256), hist_pollen_eq, color='steelblue', width=1.0)
axes[1, 1].set_title('Histograma Equalizado')
axes[1, 1].set_xlabel('Nível de cinza')
axes[1, 1].set_ylabel('Número de pixels')
axes[1, 1].set_xlim([0, 255])
```



```
plt.tight_layout()
plt.show()
```

Equalização de Histograma — Pollen



In [77]:

```
# --- Comparação para MESSI ---
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
fig.suptitle('Equalização de Histograma - Messi', fontsize=15, fontweight='bold')

# Imagem original
axes[0, 0].imshow(img_messi, cmap='gray', vmin=0, vmax=255)
axes[0, 0].set_title('Imagem Original')
axes[0, 0].axis('off')

# Histograma original
axes[0, 1].bar(range(256), messi_hist, color='gray', width=1.0)
axes[0, 1].set_title('Histograma Original')
axes[0, 1].set_xlabel('Nível de cinza')
axes[0, 1].set_ylabel('Número de pixels')
axes[0, 1].set_xlim([0, 255])

# Imagem equalizada
axes[1, 0].imshow(messi_eq, cmap='gray', vmin=0, vmax=255)
axes[1, 0].set_title('Imagem Equalizada')
axes[1, 0].axis('off')

# Histograma equalizado
hist_messi_eq = calcular_histograma(messi_eq) # calcula histograma da imagem resultado
axes[1, 1].bar(range(256), hist_messi_eq, color='steelblue', width=1.0)
axes[1, 1].set_title('Histograma Equalizado')
```

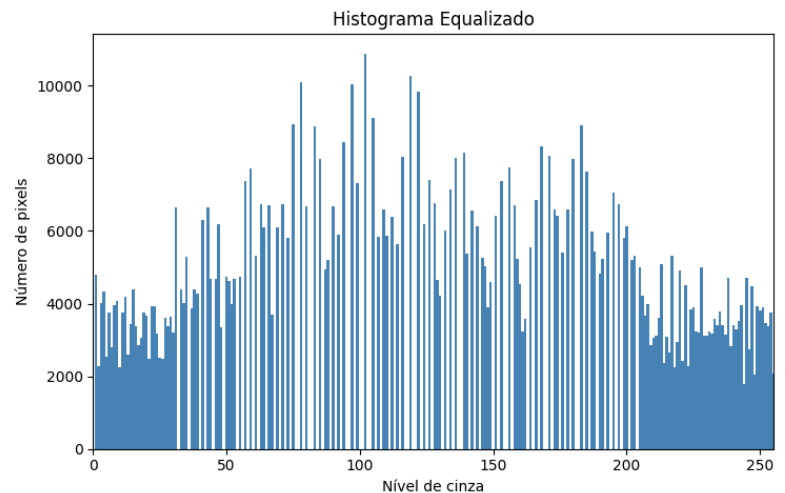
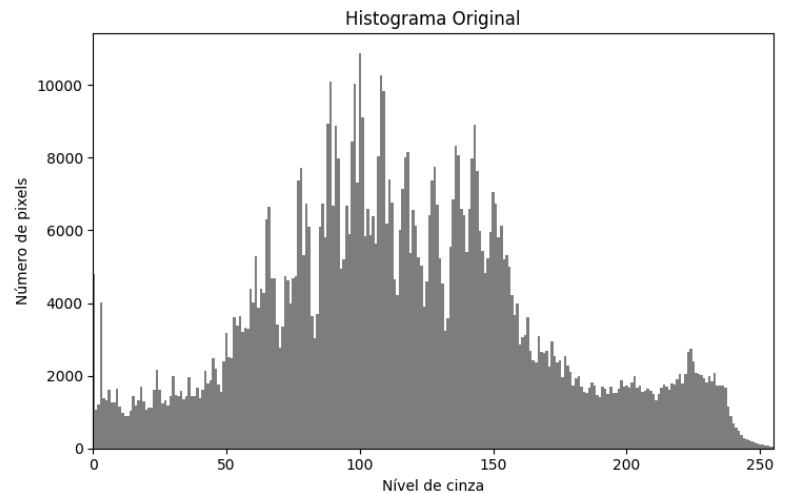
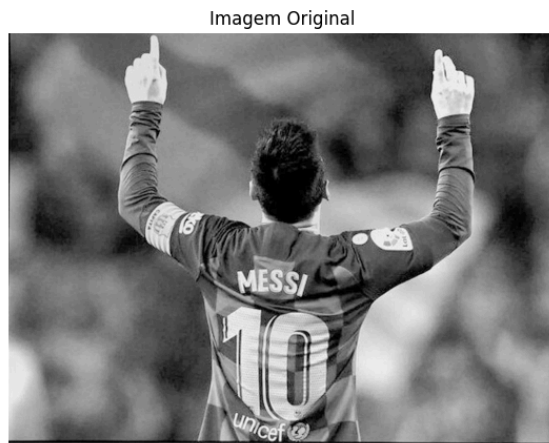
```

axes[1, 1].set_xlabel('Nível de cinza')
axes[1, 1].set_ylabel('Número de pixels')
axes[1, 1].set_xlim([0, 255])

plt.tight_layout()
plt.show()

```

Equalização de Histograma — Messi



(d) Análise: por que o histograma equalizado não fica perfeitamente plano?

O histograma equalizado nunca fica perfeitamente uniforme por dois motivos principais. Primeiro, a transformação $s_k = \text{round}((L - 1) \cdot \text{CDF}(r_k))$ é uma função discreta que mapeia 256 níveis inteiros de entrada em 256 níveis inteiros de saída; como o arredondamento faz com que vários níveis de entrada diferentes sejam mapeados para o **mesmo** nível de saída, os pixels desses níveis se acumulam em um único bin, gerando picos no histograma resultado. Segundo, e como consequência direta, os níveis de saída que não recebem nenhum mapeamento ficam vazios, produzindo os "buracos" (bins com contagem zero) observados. Em resumo, a natureza discreta dos níveis de cinza e a operação de arredondamento impedem uma redistribuição perfeitamente uniforme — o que só seria possível no caso contínuo, onde a CDF é uma função estritamente monotônica e inversível.

Parte II: Especificação de Histograma

A equalização às vezes deixa o resultado "lavado". A **especificação** permite copiar o clima de uma foto bem exposta e aplicar na sua foto problemática.

Ideia: dada uma imagem de entrada e uma imagem de referência (com bom contraste), obtemos histogramas tais que a imagem de saída tenha histograma semelhante ao da referência.

(a) Implementação da especificação de histograma

Algoritmo:

1. Calcular CDF da imagem de entrada: $T(r_k) = (L - 1) \cdot \text{CDF}_{\text{entrada}}(r_k)$.
2. Calcular CDF da imagem de referência: $G(z_k) = (L - 1) \cdot \text{CDF}_{\text{ref}}(z_k)$.
3. Para cada nível r_k da entrada, calcular $s = T(r_k)$ e encontrar o z tal que $G(z)$ é o **mais próximo** de s (vizinho mais próximo para inverter G).

4. Mapear cada pixel da imagem de entrada usando a LUT resultante.

```
In [78]: def especificar_histograma(img_entrada, img_referencia):
        """
        Realiza a especificação (matching) de histograma:
        - img_entrada: imagem cujo histograma será transformado.
        - img_referencia: imagem cujo histograma serve de modelo.
        Retorna a imagem especificada e dados intermediários.
        """
        L = 256

        # ----- CDF da imagem de ENTRADA -----
        linhas_e, colunas_e = img_entrada.shape
        num_pixels_e = linhas_e * colunas_e
        hist_e = calcular_histograma(img_entrada)
        pr_e = histograma_normalizado(hist_e, num_pixels_e)
        cdf_e = calcular_cdf(pr_e)
        # Transformação  $T(r_k) = \text{round}((L-1) * CDF_{entrada}(r_k))$ 
        T = np.round((L - 1) * cdf_e).astype(np.uint8)

        # ----- CDF da imagem de REFERÊNCIA -----
        linhas_r, colunas_r = img_referencia.shape
        num_pixels_r = linhas_r * colunas_r
        hist_r = calcular_histograma(img_referencia)
        pr_r = histograma_normalizado(hist_r, num_pixels_r)
        cdf_r = calcular_cdf(pr_r)
        # Transformação  $G(z_k) = \text{round}((L-1) * CDF_{ref}(z_k))$ 
        G = np.round((L - 1) * cdf_r).astype(np.uint8)

        # ----- Inverter G pelo vizinho mais próximo -----
        # Para cada nível s (0..255), encontrar z tal que  $|G(z) - s|$  é mínimo.
        # Isso constrói  $G^{-1}$ .
        G_inv = np.zeros(L, dtype=np.uint8)
        for s in range(L):
            min_diff = 256 # valor impossível de diferença
            melhor_z = 0
            for z in range(L):
                diff = abs(int(G[z]) - s) # diferença absoluta
                if diff < min_diff:
                    min_diff = diff
                    melhor_z = z
            G_inv[s] = melhor_z

        # ----- Montar LUT composta:  $z = G^{-1}(T(r))$  -----
        lut = np.zeros(L, dtype=np.uint8)
        for r in range(L):
            lut[r] = G_inv[T[r]] # composição  $G^{-1} \circ T$ 

        # ----- Aplicar a LUT na imagem de entrada -----
        img_spec = np.zeros_like(img_entrada)
        for i in range(linhas_e):
            for j in range(colunas_e):
                img_spec[i, j] = lut[img_entrada[i, j]]

        return img_spec, hist_e, hist_r, T, G, G_inv, lut
```

(b) Aplicação: usar `goodContrast.tif` como referência

Aplicamos o estilo do histograma da imagem de bom contraste nas duas imagens de baixo contraste da Parte I.

```
In [79]: # Carregar imagem de referência em escala de cinza
img_ref = np.array(Image.open('goodContrast.tif').convert('L'))
print(f'Referência - shape: {img_ref.shape}, dtype: {img_ref.dtype}')
```

Referência - shape: (1200, 1800), dtype: uint8

```
In [80]: # Especificação para pollen
pollen_spec, pollen_hist_spec_in, _, pollen_T, pollen_G, pollen_Ginv, pollen_lut = \
    especificar_histograma(img_pollen, img_ref)

# Especificação para messi
messi_spec, messi_hist_spec_in, _, messi_T, messi_G, messi_Ginv, messi_lut = \
    especificar_histograma(img_messi, img_ref)

print('Especificação de histograma concluída para ambas as imagens.')
```

Especificação de histograma concluída para ambas as imagens.

Comparação: Original × Referência × Especificada — Pollen

```
In [81]: # --- Comparação 3 imagens + 3 histogramas para POLLEN ---
fig, axes = plt.subplots(2, 3, figsize=(18, 10))
fig.suptitle('Especificação de Histograma - Pollen', fontsize=15, fontweight='bold')

# Linha 1: imagens
axes[0, 0].imshow(img_pollen, cmap='gray', vmin=0, vmax=255)
axes[0, 0].set_title('Original (baixo contraste)')
axes[0, 0].axis('off')

axes[0, 1].imshow(img_ref, cmap='gray', vmin=0, vmax=255)
axes[0, 1].set_title('Referência (goodContrast)')
axes[0, 1].axis('off')

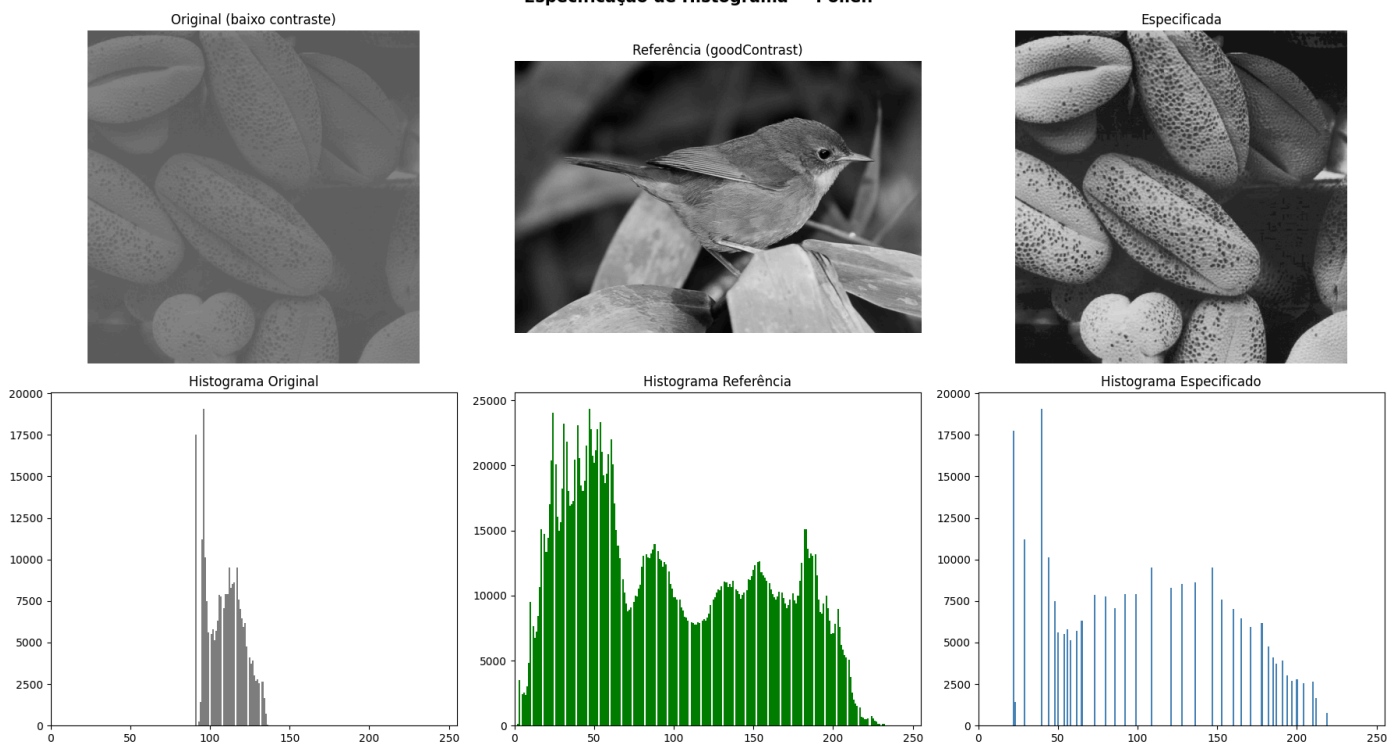
axes[0, 2].imshow(pollen_spec, cmap='gray', vmin=0, vmax=255)
axes[0, 2].set_title('Especificada')
axes[0, 2].axis('off')

# Linha 2: histogramas
axes[1, 0].bar(range(256), calcular_histograma(img_pollen), color='gray', width=1.0)
axes[1, 0].set_title('Histograma Original')
axes[1, 0].set_xlim([0, 255])

axes[1, 1].bar(range(256), calcular_histograma(img_ref), color='green', width=1.0)
axes[1, 1].set_title('Histograma Referência')
axes[1, 1].set_xlim([0, 255])

axes[1, 2].bar(range(256), calcular_histograma(pollen_spec), color='steelblue', width=1.0)
axes[1, 2].set_title('Histograma Especificado')
axes[1, 2].set_xlim([0, 255])

plt.tight_layout()
plt.show()
```



Comparação: Original × Referência × Especificada — Messi

In [82]:

```
# --- Comparação 3 imagens + 3 histogramas para MESSI ---
fig, axes = plt.subplots(2, 3, figsize=(18, 10))
fig.suptitle('Especificação de Histograma - Messi', fontsize=15, fontweight='bold')

# Linha 1: imagens
axes[0, 0].imshow(img_messi, cmap='gray', vmin=0, vmax=255)
axes[0, 0].set_title('Original (baixo contraste)')
axes[0, 0].axis('off')

axes[0, 1].imshow(img_ref, cmap='gray', vmin=0, vmax=255)
axes[0, 1].set_title('Referência (goodContrast)')
axes[0, 1].axis('off')

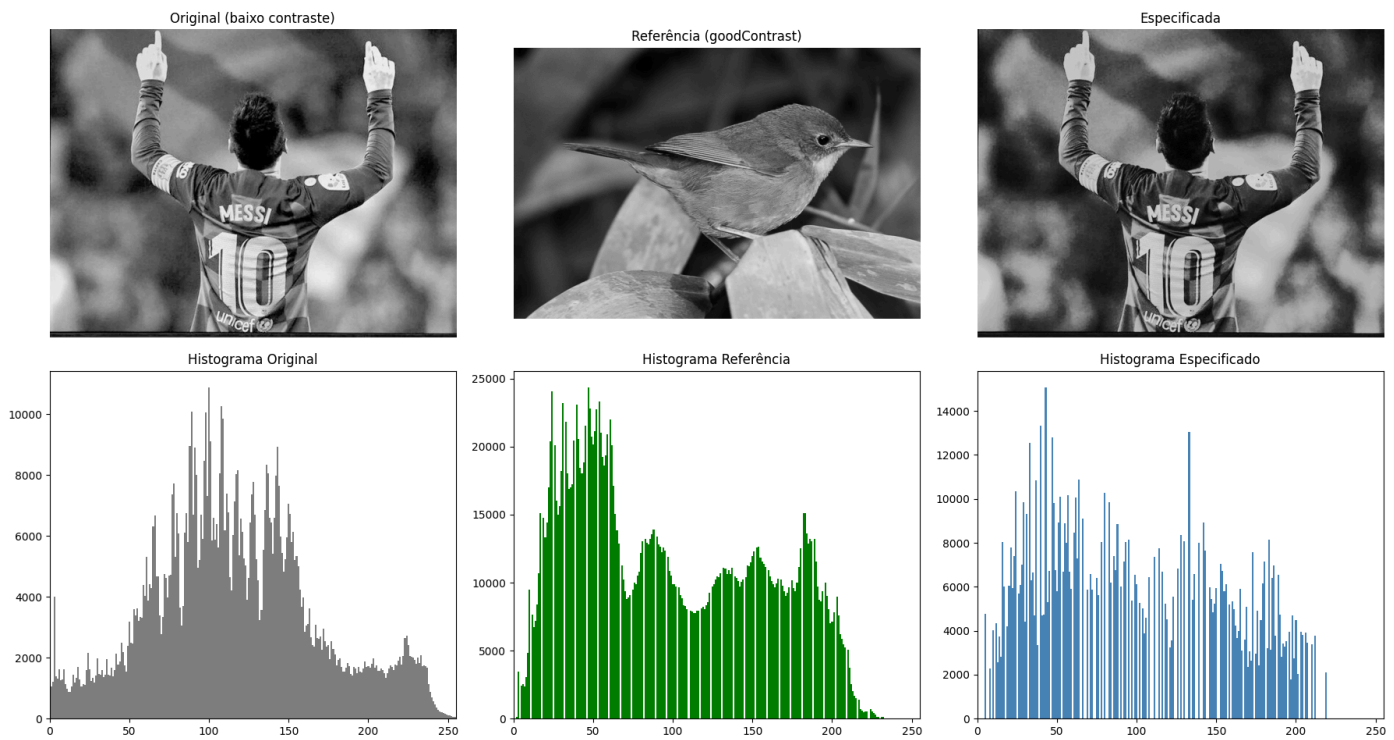
axes[0, 2].imshow(messi_spec, cmap='gray', vmin=0, vmax=255)
axes[0, 2].set_title('Especificada')
axes[0, 2].axis('off')

# Linha 2: histogramas
axes[1, 0].bar(range(256), calcular_historama(img_messi), color='gray', width=1.0)
axes[1, 0].set_title('Histograma Original')
axes[1, 0].set_xlim([0, 255])

axes[1, 1].bar(range(256), calcular_historama(img_ref), color='green', width=1.0)
axes[1, 1].set_title('Histograma Referência')
axes[1, 1].set_xlim([0, 255])

axes[1, 2].bar(range(256), calcular_historama(messi_spec), color='steelblue', width=1.0)
axes[1, 2].set_title('Histograma Especificado')
axes[1, 2].set_xlim([0, 255])

plt.tight_layout()
plt.show()
```



(c) Análise: A equalização é um caso particular da especificação?

Sim. A **equalização de histograma** é um caso particular da **especificação de histograma** em que o histograma alvo (referência) é o histograma **uniforme** (plano). Ou seja, se tomarmos como referência uma distribuição em que todos os 256 níveis de cinza possuem a mesma probabilidade $p_z(z_k) = 1/L$ para todo k , a CDF da referência torna-se $G(z_k) = (k + 1)/L$, que é exatamente uma reta. Ao inverter G e compor com T , recuperamos a transformação de equalização clássica

$s_k =$

$\text{textround}((L - 1)$

$\cdot$

$\text{textCDF}_{\text{textentrada}}(r_k))$

. Portanto, a equalização é simplesmente a especificação com histograma-alvo uniforme.

Questão 2: Construindo a lente. Filtros lineares homogêneos.

Objetivo: implementar do zero o filtro de média / Gaussiano e estudar máscara, tamanho e tratamento de borda.

Parte A — A máscara e a janela deslizante

(a) Montagem dos kernels (média e Gaussiano)

- **Média:** pesos uniformes $1/k^2$.
- **Gaussiano 2D:** $h(x, y) = \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right)$, normalizado para soma 1.
- Pares (k, σ) : (3, 0.5), (7, 1.2), (15, 2.5).

```
In [83]: import math

def kernel_media(k):
    """
    Cria kernel de média k×k com pesos uniformes 1/k².
    """
    return np.ones((k, k), dtype=np.float64) / (k * k)

def kernel_gaussiano(k, sigma):
    """
    Cria kernel Gaussiano 2D k×k.
    h(x,y) = exp(-(x²+y²) / (2σ²)), depois normaliza pela soma.
```

```

"""
r = (k - 1) // 2                                # raio da janela
mask = np.zeros((k, k), dtype=np.float64)
for i in range(k):
    for j in range(k):
        x = i - r                                # coordenada centrada
        y = j - r
        mask[i, j] = math.exp(-(x*x + y*y) / (2.0 * sigma * sigma))
mask /= mask.sum()                               # normaliza para soma = 1
return mask

# Parâmetros exigidos
configs = [(3, 0.5), (7, 1.2), (15, 2.5)]

print('=== Kernels de Média ===')
kernels_media = {}
for k, _ in configs:
    km = kernel_media(k)
    kernels_media[k] = km
    assert abs(km.sum() - 1.0) < 1e-10, f'Soma do kernel média {k}x{k} != 1'
    assert np.allclose(km, km.T), f'Kernel média {k}x{k} não é simétrico'
    print(f' k={k}: soma = {km.sum():.15f}, simétrico = {np.allclose(km, km.T)}')

print()
print('=== Kernels Gaussianos ===')
kernels_gauss = {}
for k, sigma in configs:
    kg = kernel_gaussiano(k, sigma)
    kernels_gauss[k] = kg
    assert abs(kg.sum() - 1.0) < 1e-10, f'Soma do kernel Gauss {k}x{k} != 1'
    assert np.allclose(kg, kg.T), f'Kernel Gauss {k}x{k} não é simétrico'
    print(f' k={k}, σ={sigma}: soma = {kg.sum():.15f}, simétrico = {np.allclose(kg, kg.T)}')

```

```

=== Kernels de Média ===
k=3: soma = 1.000000000000000, simétrico = True
k=7: soma = 1.000000000000000, simétrico = True
k=15: soma = 1.000000000000000, simétrico = True

=== Kernels Gaussianos ===
k=3, σ=0.5: soma = 1.000000000000000, simétrico = True
k=7, σ=1.2: soma = 1.000000000000000, simétrico = True
k=15, σ=2.5: soma = 1.000000000000000, simétrico = True

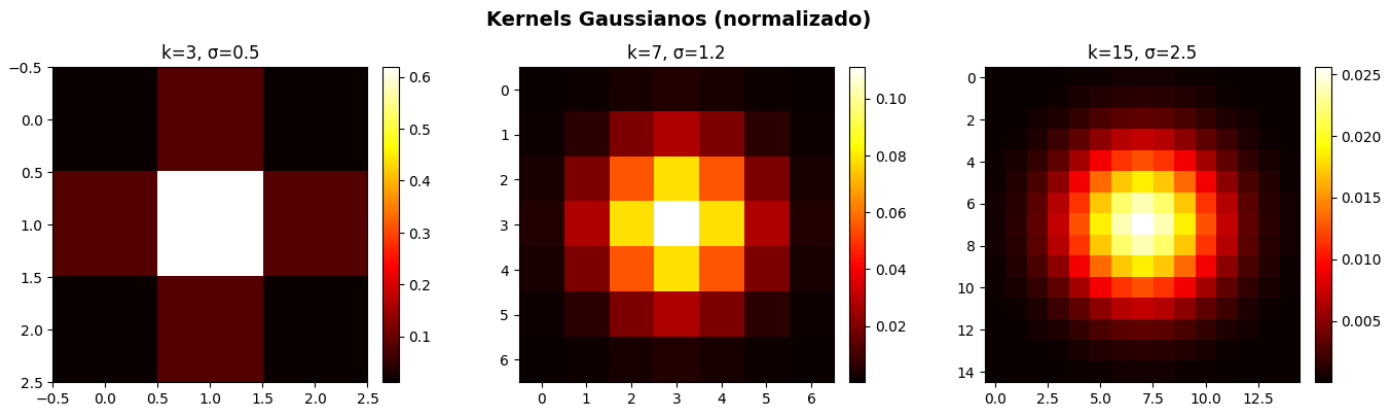
```

```

In [84]: # Visualização dos kernels Gaussianos como imagens

fig, axes = plt.subplots(1, 3, figsize=(14, 4))
fig.suptitle('Kernels Gaussianos (normalizado)', fontsize=14, fontweight='bold')
for idx, (k, sigma) in enumerate(configs):
    im = axes[idx].imshow(kernels_gauss[k], cmap='hot', interpolation='nearest')
    axes[idx].set_title(f'k={k}, σ={sigma}')
    plt.colorbar(im, ax=axes[idx], fraction=0.046, pad=0.04)
plt.tight_layout()
plt.show()

```

(b) Motor: janela deslizante com `np.pad`

Implementação com dois `for` aninhados. Suporta `modo` de padding (`'zeros'` ou `'reflect'`).

```
In [85]: def filtro_linear(img, kernel, modo='zeros'):
    """
    Aplica filtro linear (convolução) usando janela deslizante.
    img      : imagem de entrada (2D, uint8 ou float)
    kernel   : máscara 2D (k×k)
    modo     : 'zeros' → padding de zeros; 'reflect' → padding por reflexão
    Retorna imagem filtrada (uint8).
    """

    k = kernel.shape[0]      # tamanho da janela (assume quadrada)
    r = k // 2               # raio

    # Escolher modo de padding do np.pad
    if modo == 'zeros':
        pad_mode = 'constant'    # preenche com 0 por padrão
        img_pad = np.pad(img.astype(np.float64), r, mode=pad_mode, constant_values=0)
    elif modo == 'reflect':
        pad_mode = 'reflect'
        img_pad = np.pad(img.astype(np.float64), r, mode=pad_mode)
    else:
        raise ValueError(f'Modo desconhecido: {modo}')

    linhas, colunas = img.shape
    saida = np.zeros((linhas, colunas), dtype=np.float64)

    # Janela deslizante: dois for aninhados
    for i in range(linhas):
        for j in range(colunas):
            # Extrair vizinhança k×k centrada em (i, j) da imagem com padding
            vizinhanca = img_pad[i:i+k, j:j+k]
            # Produto elemento-a-elemento e soma (correlação, mas kernel simétrico)
            saida[i, j] = np.sum(vizinhanca * kernel)

    # Recortar para [0, 255] e converter para uint8
    saida = np.clip(saida, 0, 255).astype(np.uint8)
    return saida
```

Parte B — Testando a lente

```
In [86]: # Carregar imagens de teste
img_circuit = np.array(Image.open('circuit.tif').convert('L'))
```



```
img_caractere = np.array(Image.open('caractere.tif').convert('L'))
```

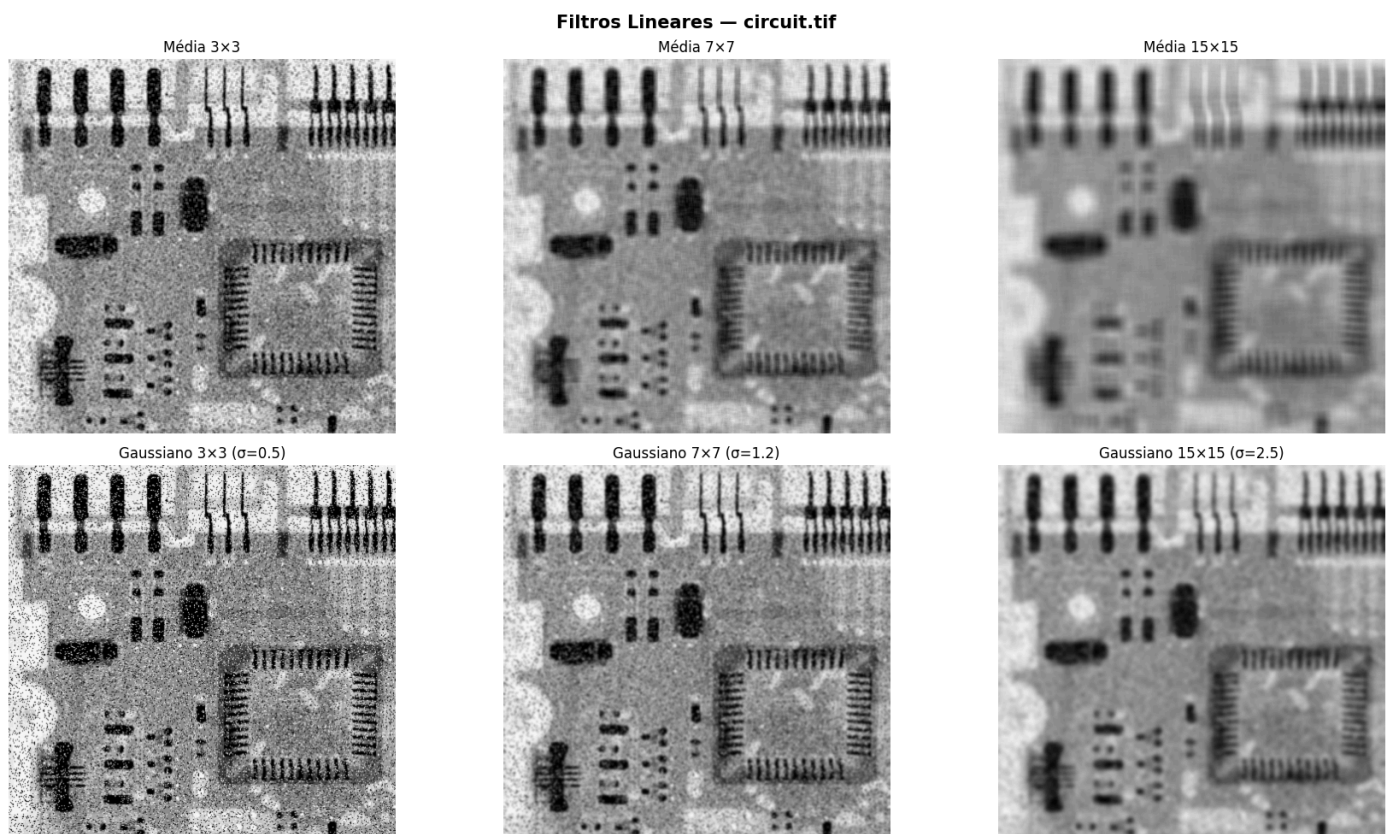
```
print(f'circuit - shape: {img_circuit.shape}')  
print(f'caractere - shape: {img_caractere.shape}')
```

```
circuit - shape: (440, 455)  
caractere - shape: (500, 500)
```

(a) Aplicação: Média × Gaussiano para $k \in \{3, 7, 15\}$

6 saídas em grid para cada imagem de teste.

```
In [87]: # --- Grid para circuit.tif ---  
fig, axes = plt.subplots(2, 3, figsize=(18, 10))  
fig.suptitle('Filtros Lineares - circuit.tif', fontsize=15, fontweight='bold')  
  
for col_idx, (k, sigma) in enumerate(configs):  
    # Média  
    res_media = filtro_linear(img_circuit, kernels_media[k], modo='reflect')  
    axes[0, col_idx].imshow(res_media, cmap='gray', vmin=0, vmax=255)  
    axes[0, col_idx].set_title(f'Média {k}x{k}')  
    axes[0, col_idx].axis('off')  
  
    # Gaussiano  
    res_gauss = filtro_linear(img_circuit, kernels_gauss[k], modo='reflect')  
    axes[1, col_idx].imshow(res_gauss, cmap='gray', vmin=0, vmax=255)  
    axes[1, col_idx].set_title(f'Gaussiano {k}x{k} (σ={sigma})')  
    axes[1, col_idx].axis('off')  
  
plt.tight_layout()  
plt.show()
```



```
In [88]: # --- Grid para caractere.tif ---  
fig, axes = plt.subplots(2, 3, figsize=(18, 10))  
fig.suptitle('Filtros Lineares - caractere.tif', fontsize=15, fontweight='bold')
```

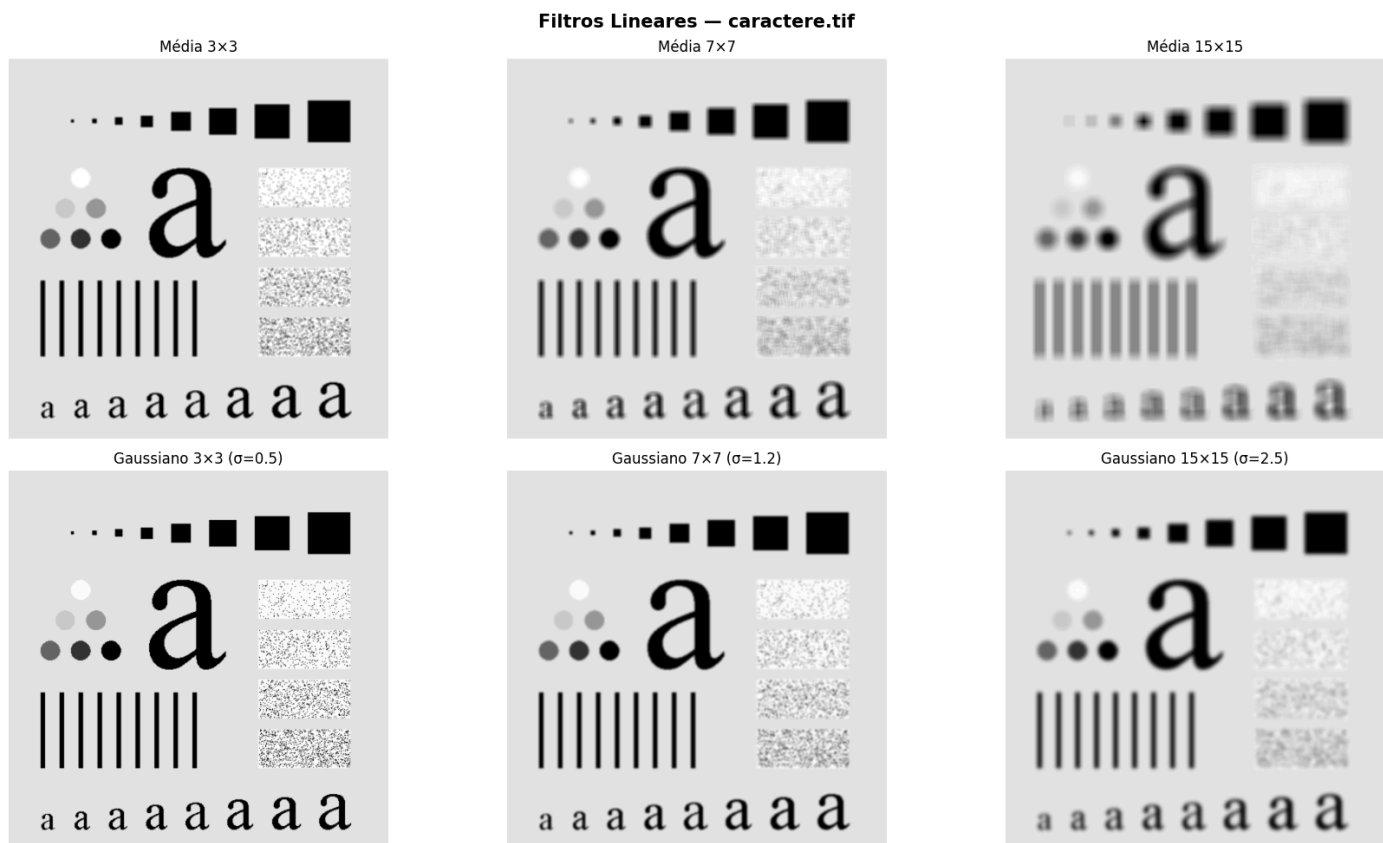
```

for col_idx, (k, sigma) in enumerate(configs):
    # Média
    res_media = filtro_linear(img_caractere, kernels_media[k], modo='reflect')
    axes[0, col_idx].imshow(res_media, cmap='gray', vmin=0, vmax=255)
    axes[0, col_idx].set_title(f'Média {k}x{k}')
    axes[0, col_idx].axis('off')

    # Gaussiano
    res_gauss = filtro_linear(img_caractere, kernels_gauss[k], modo='reflect')
    axes[1, col_idx].imshow(res_gauss, cmap='gray', vmin=0, vmax=255)
    axes[1, col_idx].set_title(f'Gaussiano {k}x{k} (σ={sigma})')
    axes[1, col_idx].axis('off')

plt.tight_layout()
plt.show()

```



(b) Bordas: padding de zeros vs. reflexão (k=15)

Comparação visual com zoom na borda da imagem.

```

In [89]: # Aplicar Gaussiano 15×15 com os dois modos de padding
k_borda = 15
sigma_borda = 2.5
kg15 = kernels_gauss[k_borda]

res_zeros = filtro_linear(img_circuit, kg15, modo='zeros')
res_reflect = filtro_linear(img_circuit, kg15, modo='reflect')

# Região de zoom na borda (canto superior esquerdo)
zoom = 40 # pixels de borda a mostrar

fig, axes = plt.subplots(2, 3, figsize=(18, 10))
fig.suptitle('Comparação de Padding — Gaussiano 15×15 (circuit.tif)', fontsize=14, fontweight='bold')

# Imagem completa

```

```

axes[0, 0].imshow(img_circuit, cmap='gray', vmin=0, vmax=255)
axes[0, 0].set_title('Original')
axes[0, 0].axis('off')

axes[0, 1].imshow(res_zeros, cmap='gray', vmin=0, vmax=255)
axes[0, 1].set_title('Padding Zeros')
axes[0, 1].axis('off')

axes[0, 2].imshow(res_reflect, cmap='gray', vmin=0, vmax=255)
axes[0, 2].set_title('Padding Reflexão')
axes[0, 2].axis('off')

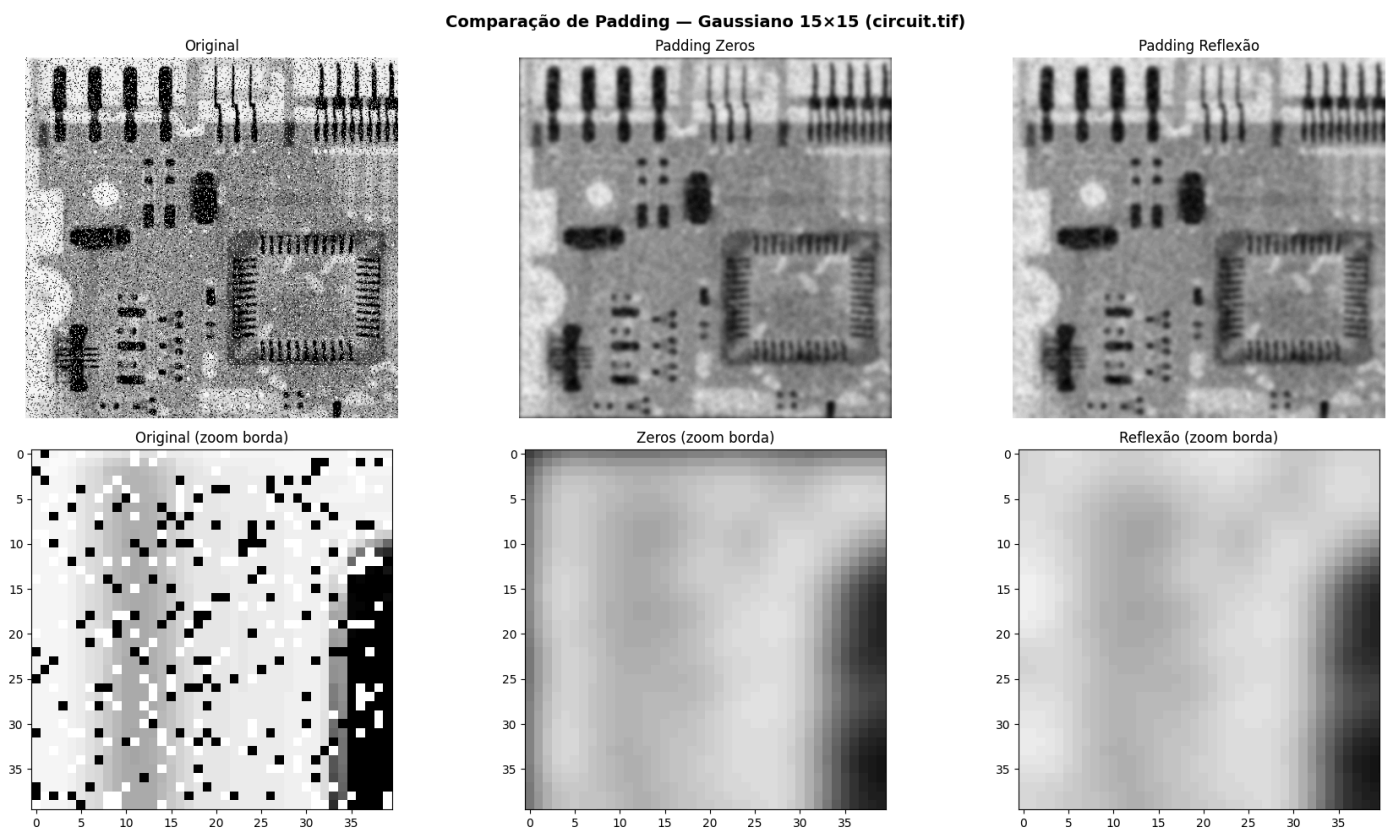
# Zoom na borda
axes[1, 0].imshow(img_circuit[:zoom, :zoom], cmap='gray', vmin=0, vmax=255)
axes[1, 0].set_title('Original (zoom borda)')

axes[1, 1].imshow(res_zeros[:zoom, :zoom], cmap='gray', vmin=0, vmax=255)
axes[1, 1].set_title('Zeros (zoom borda)')

axes[1, 2].imshow(res_reflect[:zoom, :zoom], cmap='gray', vmin=0, vmax=255)
axes[1, 2].set_title('Reflexão (zoom borda)')

plt.tight_layout()
plt.show()

```



Explicação da diferença visual nas bordas:

Com **padding de zeros**, os pixels próximos à borda da imagem são calculados utilizando vizinhos artificiais com valor 0 (preto). Isso faz com que a média ponderada nessas regiões seja puxada para baixo, resultando em um **escurecimento visível** nas bordas da imagem filtrada — um artefato indesejado.

Com **padding por reflexão** (`reflect`), os pixels fora da borda são preenchidos espelhando os pixels internos. Isso garante que a vizinhança do filtro contenha valores coerentes com o conteúdo real da imagem, produzindo bordas **muito mais naturais**, sem o artefato de escurecimento. Por isso, a reflexão é geralmente preferida na prática.

(c) Desempenho: tempo do Gaussiano 15×15

```
In [90]: import time

# Medir tempo do Gaussiano 15x15 na imagem circuit
t_inicio = time.time()
_ = filtro_linear(img_circuit, kernels_gauss[15], modo='reflect')
t_fim = time.time()

print(f'Gaussiano 15x15 em circuit.tif ({img_circuit.shape}): {t_fim - t_inicio:.2f} s')

# Medir tempo na imagem caractere
t_inicio = time.time()
_ = filtro_linear(img_caractere, kernels_gauss[15], modo='reflect')
t_fim = time.time()

print(f'Gaussiano 15x15 em caractere.tif ({img_caractere.shape}): {t_fim - t_inicio:.2f} s')
```

```
Gaussiano 15x15 em circuit.tif ((440, 455)): 0.98 s
Gaussiano 15x15 em caractere.tif ((500, 500)): 1.23 s
```

(d) Análise: para $k=15$, qual filtro preserva mais as bordas?

Para o mesmo tamanho de janela $k = 15$, o **filtro Gaussiano** preserva melhor as bordas da imagem do que o filtro de **média**. A razão está na distribuição dos pesos: no filtro de média, todos os $k^2 = 225$ pixels da vizinhança contribuem igualmente ($1/225$), o que causa um borramento uniforme e intenso — inclusive misturando pixels de regiões distintas (por exemplo, os dois lados de uma aresta). Já no filtro Gaussiano, os pesos decaem exponencialmente com a distância ao centro. Assim, os pixels mais próximos do pixel central têm influência muito maior, enquanto os pixels distantes (que podem estar do outro lado de uma borda) contribuem de forma quase desprezível. Esse decaimento suave permite suavizar o ruído sem destruir completamente as transições abruptas de intensidade que definem as bordas da imagem.

Questão 3 — Modo retrato. Filtro espacialmente variável.

Objetivo: manter o assunto principal nítido e desfocar o fundo, simulando profundidade de campo fotográfica (efeito Bokeh / Tilt-Shift).

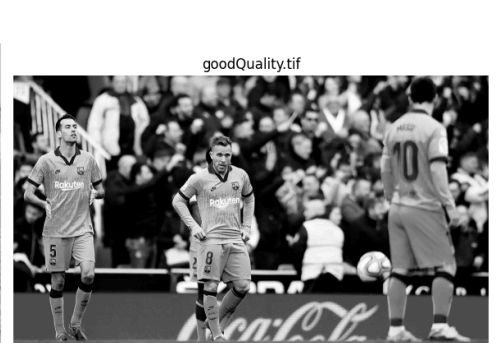
Parte Única — Simulação de profundidade de campo

```
In [91]: # Carregar as três imagens em escala de cinza
img_loco = np.array(Image.open('locomotive.png').convert('L'))
img_bee = np.array(Image.open('BumbleBee.png').convert('L'))
img_good = np.array(Image.open('goodQuality.tif').convert('L'))

print(f'locomotive - shape: {img_loco.shape}')
print(f'BumbleBee - shape: {img_bee.shape}')
print(f'goodQuality - shape: {img_good.shape}')

# Mostrar as três originais
fig, axes = plt.subplots(1, 3, figsize=(18, 5))
for ax, im, nome in zip(axes, [img_loco, img_bee, img_good],
                          ['locomotive.png', 'BumbleBee.png', 'goodQuality.tif']):
    ax.imshow(im, cmap='gray', vmin=0, vmax=255)
    ax.set_title(nome)
    ax.axis('off')
plt.tight_layout()
plt.show()
```

```
locomotive - shape: (1956, 1756)
BumbleBee - shape: (1850, 2650)
goodQuality - shape: (1080, 1920)
```



(a) Máscara matemática R

Criamos duas variantes:

- **Círculo** (modo retrato): 1 dentro de um raio ao redor do centro, 0 fora.
- **Faixa horizontal** (efeito tilt-shift): 1 em uma faixa horizontal central, 0 fora.

(b) Máscara suavizada $\tilde{R}$

Aplicamos o filtro Gaussiano da Questão 2 na máscara dura R para criar um **degradê suave** nas bordas, evitando o efeito de "adesivo colado".

```
In [92]: def mascara_circulo(shape, centro=None, raio=None):
    """
    Cria máscara binária com 1 dentro do círculo e 0 fora.
    shape : (linhas, colunas) da imagem
    centro : (cy, cx) – padrão: centro da imagem
    raio : raio do círculo – padrão: 1/4 da menor dimensão
    """
    linhas, colunas = shape
    if centro is None:
        centro = (linhas // 2, colunas // 2)
    if raio is None:
        raio = min(linhas, colunas) // 4

    R = np.zeros(shape, dtype=np.float64)
    cy, cx = centro
    for i in range(linhas):
        for j in range(colunas):
            dist = math.sqrt((i - cy)**2 + (j - cx)**2)
            if dist <= raio:
                R[i, j] = 1.0
    return R

def mascara_faixa(shape, y_centro=None, largura=None):
    """
    Cria máscara binária com 1 em uma faixa horizontal e 0 fora.
    shape : (linhas, colunas)
    y_centro : linha central da faixa – padrão: centro vertical
    largura : metade da largura da faixa – padrão: 1/6 das linhas
    """
    linhas, colunas = shape
    if y_centro is None:
        y_centro = linhas // 2
    if largura is None:
        largura = linhas // 6
```



```

R = np.zeros(shape, dtype=np.float64)
for i in range(linhas):
    if abs(i - y_centro) <= largura:
        R[i, :] = 1.0
return R

def suavizar_mascara(R, k_suav, sigma_suav):
    """
    Suaviza a máscara R aplicando o filtro Gaussiano, gerando  $\tilde{R}$ .
    O resultado é normalizado para ficar em [0, 1].
    """
    kg = kernel_gaussiano(k_suav, sigma_suav)
    # Converter R para uint8 temporariamente (0 ou 255) para usar filtro_linear
    R_u8 = (R * 255).astype(np.uint8)
    R_suav_u8 = filtro_linear(R_u8, kg, modo='reflect')
    # Converter de volta para [0, 1]
    R_suav = R_suav_u8.astype(np.float64) / 255.0
    return R_suav

```

(c) Fundo borrado $\tilde{I}$

Geramos a versão completamente desfocada da imagem com o filtro Gaussiano.

(d) Mesclagem

$$I_{\text{final}} = \tilde{R} \cdot I + (1 - \tilde{R}) \cdot \tilde{I}$$

Mostramos os passos intermediários: R , $\tilde{R}$, $\tilde{I}$ e o resultado final.

(e) Aplicação nas três imagens

```

In [93]: def efeito_bokeh(img, R, k_blur, sigma_blur, k_suav, sigma_suav, titulo=''):
    """
    Pipeline completo do efeito Bokeh/Tilt-Shift:
    1. Suaviza a máscara R →  $\tilde{R}$ 
    2. Desfoca a imagem inteira →  $\tilde{I}$ 
    3. Mescla:  $I_{\text{final}} = \tilde{R} \cdot I + (1 - \tilde{R}) \cdot \tilde{I}$ 
    Exibe os passos intermediários.
    """
    # (b) Máscara suavizada
    R_suav = suavizar_mascara(R, k_suav, sigma_suav)

    # (c) Fundo borrado (imagem inteira desfocada)
    kg_blur = kernel_gaussiano(k_blur, sigma_blur)
    I_blur = filtro_linear(img, kg_blur, modo='reflect')

    # (d) Mesclagem
    I_final = R_suav * img.astype(np.float64) + (1.0 - R_suav) * I_blur.astype(np.float64)
    I_final = np.clip(I_final, 0, 255).astype(np.uint8)

    # Exibir passos intermediários
    fig, axes = plt.subplots(2, 3, figsize=(18, 10))
    fig.suptitle(f'Efeito Bokeh - {titulo} (blur {k_blur}x{k_blur},  $\sigma_{\text{blur}}$ ={sigma_blur})',
                 fontsize=14, fontweight='bold')

    # Original

```

```

axes[0, 0].imshow(img, cmap='gray', vmin=0, vmax=255)
axes[0, 0].set_title('Original I')
axes[0, 0].axis('off')

# Máscara dura R
axes[0, 1].imshow(R, cmap='gray', vmin=0, vmax=1)
axes[0, 1].set_title('Máscara R (dura)')
axes[0, 1].axis('off')

# Máscara suavizada R̃
axes[0, 2].imshow(R_suav, cmap='gray', vmin=0, vmax=1)
axes[0, 2].set_title('Máscara R̃ (suave)')
axes[0, 2].axis('off')

# Imagem desfocada Î
axes[1, 0].imshow(I_blur, cmap='gray', vmin=0, vmax=255)
axes[1, 0].set_title(f'Desfocada Î (σ={sigma_blur})')
axes[1, 0].axis('off')

# Resultado final
axes[1, 1].imshow(I_final, cmap='gray', vmin=0, vmax=255)
axes[1, 1].set_title('Resultado Final')
axes[1, 1].axis('off')

# Esconder o sexto subplot
axes[1, 2].axis('off')

plt.tight_layout()
plt.show()

return I_final

```

Redimensionamento para viabilidade

Como o filtro usa loops Python, imagens muito grandes tornam o processamento extremamente lento. Redimensionamos para no máximo 300 pixels na maior dimensão, mantendo a proporção.

```

In [94]: def redimensionar(img, max_dim=300):
        """
        Redimensiona a imagem mantendo proporção para que a maior
        dimensão não ultrapasse max_dim pixels.
        Usa PIL para o resize (não é filtro, é apenas pré-processamento).
        """
        h, w = img.shape
        if max(h, w) <= max_dim:
            return img
        escala = max_dim / max(h, w)
        novo_w = int(w * escala)
        novo_h = int(h * escala)
        pil_img = Image.fromarray(img)
        pil_img = pil_img.resize((novo_w, novo_h), Image.LANCZOS)
        return np.array(pil_img)

# Redimensionar as três imagens
img_loco_sm = redimensionar(img_loco)
img_bee_sm = redimensionar(img_bee)
img_good_sm = redimensionar(img_good)

```

```
print(f'locomotive redim - shape: {img_loco_sm.shape}')
print(f'BumbleBee redim - shape: {img_bee_sm.shape}')
print(f'goodQuality redim - shape: {img_good_sm.shape}')
```

```
locomotive redim - shape: (300, 269)
BumbleBee redim - shape: (209, 300)
goodQuality redim - shape: (168, 300)
```

locomotive.png — Modo retrato (máscara circular), dois valores de σ

In [95]:

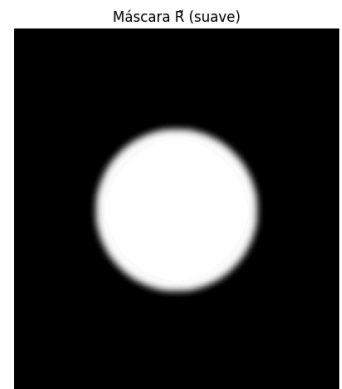
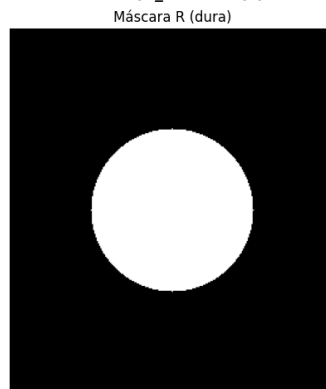
```
# Máscara circular centrada na Locomotiva
R_loco = mascara_circulo(img_loco_sm.shape,
                          centro=(img_loco_sm.shape[0]//2, img_loco_sm.shape[1]//2),
                          raio=min(img_loco_sm.shape)//4)

#  $\sigma$  pequeno → borrão Leve
print('Aplicando  $\sigma_{blur}=1.5$  ...')
_ = efeito_bokeh(img_loco_sm, R_loco,
                 k_blur=7, sigma_blur=1.5,
                 k_suav=15, sigma_suav=3.0,
                 titulo='locomotive ( $\sigma_{blur}=1.5$ )')

#  $\sigma$  grande → borrão forte
print('Aplicando  $\sigma_{blur}=3.0$  ...')
_ = efeito_bokeh(img_loco_sm, R_loco,
                 k_blur=15, sigma_blur=3.0,
                 k_suav=15, sigma_suav=3.0,
                 titulo='locomotive ( $\sigma_{blur}=3.0$ )')
```

Aplicando $\sigma_{blur}=1.5$...

Efeito Bokeh — locomotive ($\sigma_{blur}=1.5$) (blur 7×7, $\sigma_{blur}=1.5$)

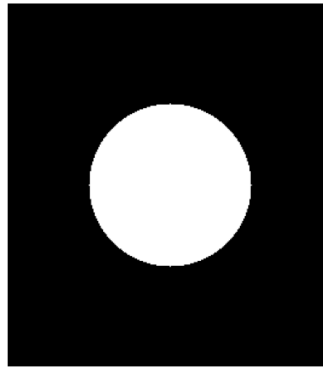


Aplicando $\sigma_{blur}=3.0$...

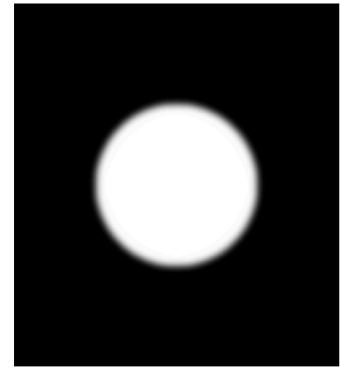
Original I



Máscara R (dura)



Máscara R (suave)

Desfocada $\tilde{I}$ ($\sigma=3.0$)

Resultado Final



BumbleBee.png — Modo retrato (máscara circular), dois valores de σ

In [96]:

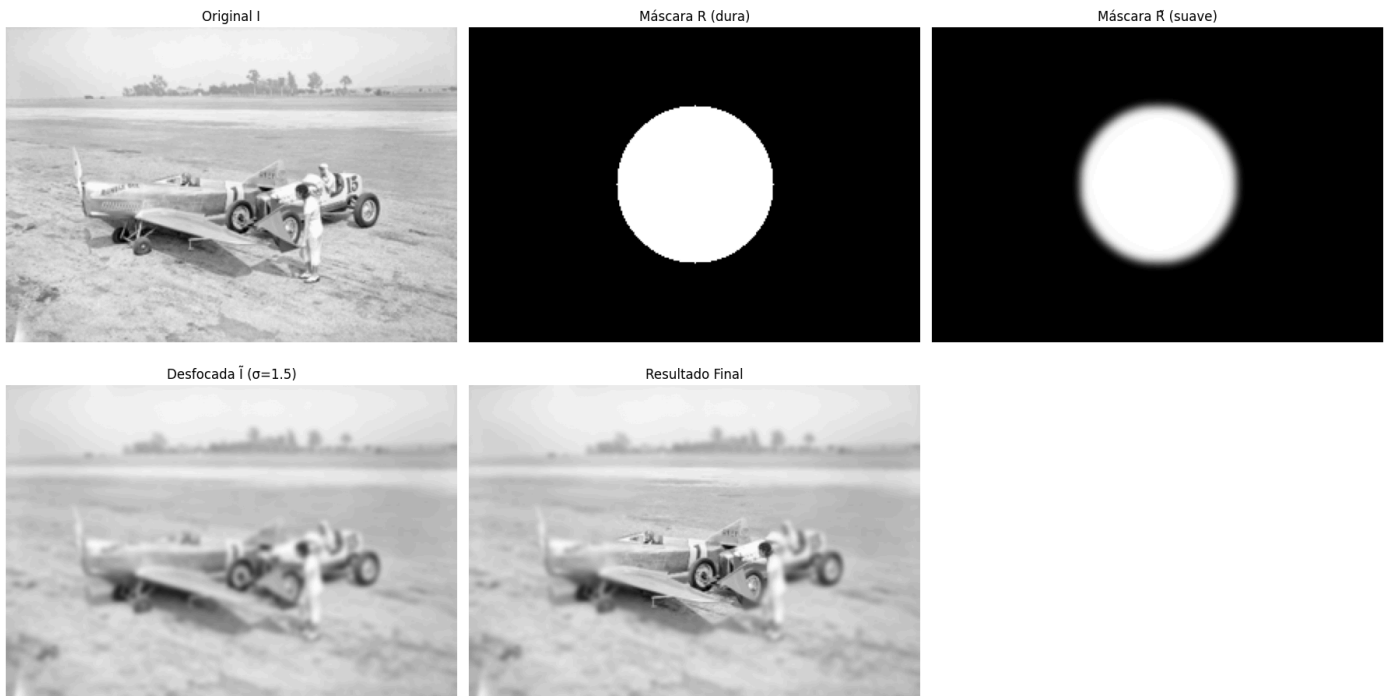
```
# Máscara circular centrada na abelha
R_bee = mascara_circulo(img_bee_sm.shape,
                        centro=(img_bee_sm.shape[0]//2, img_bee_sm.shape[1]//2),
                        raio=min(img_bee_sm.shape)//4)

#  $\sigma$  pequeno
print('Aplicando  $\sigma_{\text{blur}}=1.5$  ...')
_ = efeito_bokeh(img_bee_sm, R_bee,
                 k_blur=7, sigma_blur=1.5,
                 k_suav=15, sigma_suav=3.0,
                 titulo='BumbleBee ( $\sigma_{\text{blur}}=1.5$ )')

#  $\sigma$  grande
print('Aplicando  $\sigma_{\text{blur}}=3.0$  ...')
_ = efeito_bokeh(img_bee_sm, R_bee,
                 k_blur=15, sigma_blur=3.0,
                 k_suav=15, sigma_suav=3.0,
                 titulo='BumbleBee ( $\sigma_{\text{blur}}=3.0$ )')
```

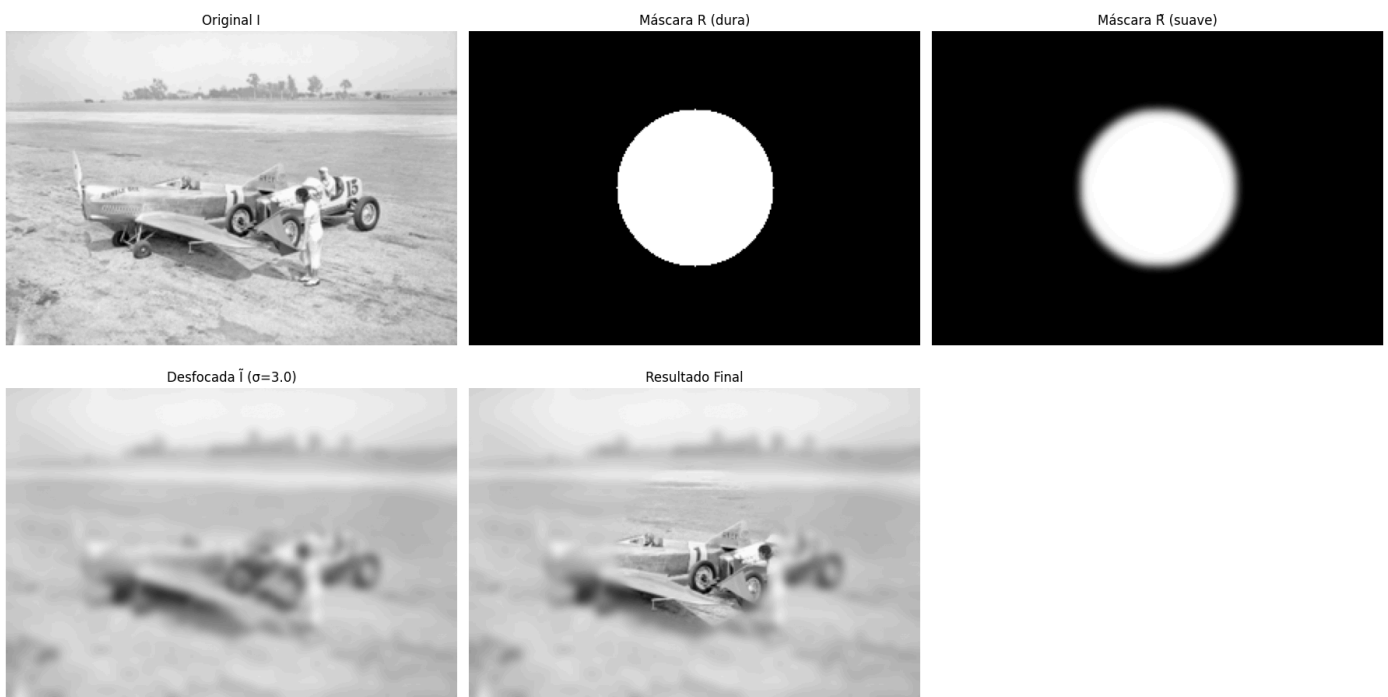
Aplicando $\sigma_{\text{blur}}=1.5$...

Efeito Bokeh — BumbleBee ($\sigma_{\text{blur}}=1.5$) (blur 7x7, $\sigma_{\text{blur}}=1.5$)



Aplicando $\sigma_{\text{blur}}=3.0$...

Efeito Bokeh — BumbleBee ($\sigma_{\text{blur}}=3.0$) (blur 15x15, $\sigma_{\text{blur}}=3.0$)



goodQuality.tif — Efeito Tilt-Shift (máscara em faixa horizontal), dois valores de σ

```
In [97]: # Máscara de faixa horizontal no centro
R_good = mascara_faixa(img_good_sm.shape,
                        y_centro=img_good_sm.shape[0]//2,
                        largura=img_good_sm.shape[0]//6)

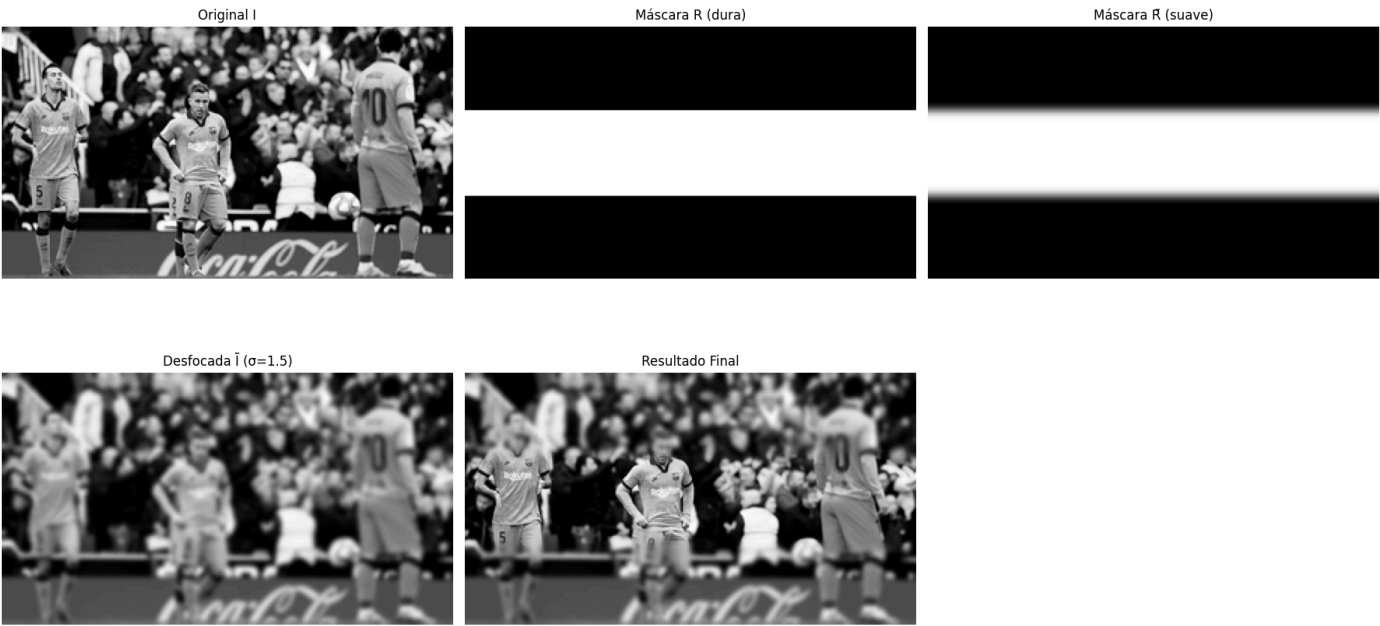
#  $\sigma$  pequeno
print('Aplicando  $\sigma_{\text{blur}}=1.5$  ...')
_ = efeito_bokeh(img_good_sm, R_good,
                 k_blur=7, sigma_blur=1.5,
                 k_suav=15, sigma_suav=3.0,
                 titulo='goodQuality tilt-shift ( $\sigma_{\text{blur}}=1.5$ )')

#  $\sigma$  grande
print('Aplicando  $\sigma_{\text{blur}}=3.0$  ...')
```

```
_ = efeito_bokeh(img_good_sm, R_good,
                 k_blur=15, sigma_blur=3.0,
                 k_suav=15, sigma_suav=3.0,
                 titulo='goodQuality tilt-shift ( $\sigma_{blur}=3.0$ )')
```

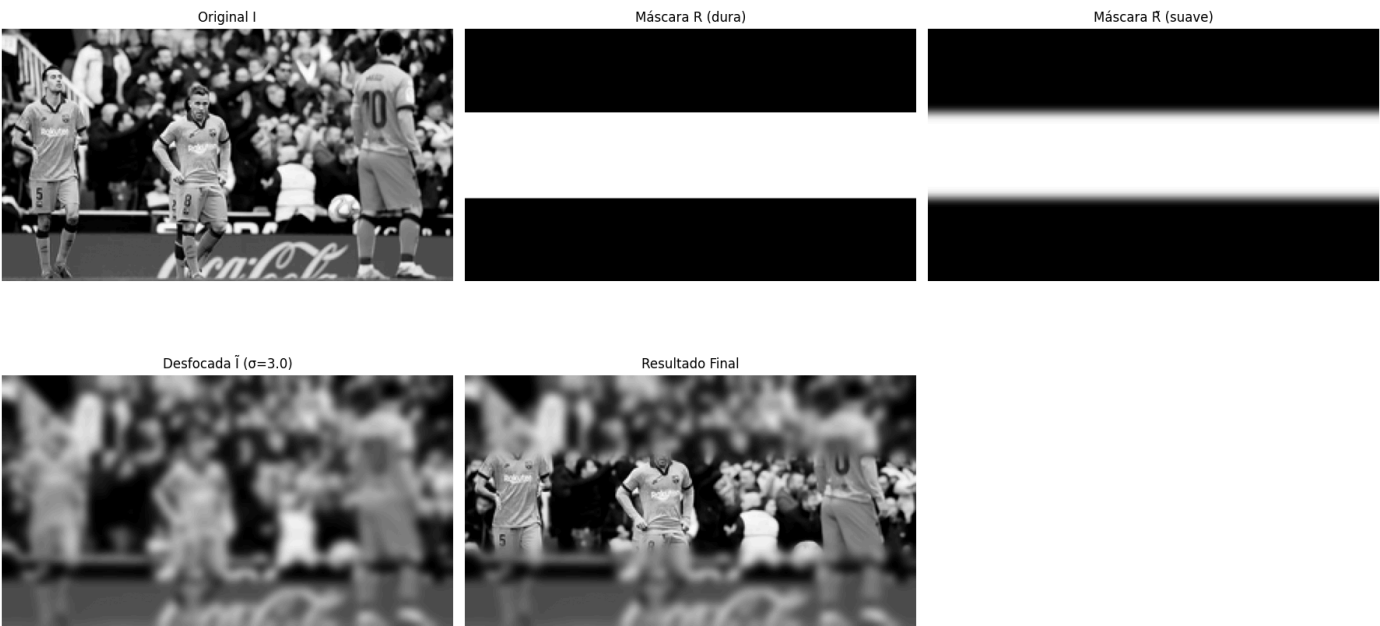
Aplicando $\sigma_{blur}=1.5$...

Efeito Bokeh — goodQuality tilt-shift ($\sigma_{blur}=1.5$) (blur 7×7, $\sigma_{blur}=1.5$)



Aplicando $\sigma_{blur}=3.0$...

Efeito Bokeh — goodQuality tilt-shift ($\sigma_{blur}=3.0$) (blur 15×15, $\sigma_{blur}=3.0$)



Comentário sobre os dois valores de σ

Ao comparar os resultados com $\sigma_{\text{blur}} = 1.5$ e $\sigma_{\text{blur}} = 3.0$, observamos que:

- Com **σ pequeno (1.5)**, o fundo fica apenas levemente desfocado. A separação entre o assunto em foco e o fundo é sutil, simulando uma abertura moderada (como f/4 ou f/5.6). O efeito é discreto e mais natural para cenas com pouca profundidade.
- Com **σ grande (3.0)**, o borrão do fundo é significativamente mais intenso, simulando uma abertura grande (como f/1.4 ou f/1.8). O assunto principal se destaca muito mais do fundo, criando um efeito Bokeh mais dramático. Porém, se a transição da máscara $\tilde{R}$ não for suficientemente suave, pode aparecer um halo artificial ao redor do objeto em foco.

Em ambos os casos, a etapa de **suavização da máscara** ($R \rightarrow \tilde{R}$) é fundamental: sem ela, a borda entre a região nítida e a desfocada seria um corte abrupto, dando a impressão de uma colagem grosseira em vez de um degradê natural de profundidade de campo.